PYTHON ACTIVITY BASED LEARNING

Algorithms • Flowcharts • Sample Programs • Outputs

Repository	Activity-Based-Learning-Python
Prepared for	Python practical / activity-based learning
Programs covered	9 activity examples based on the filenames visible in the repository context

Documentation note: The repository page could not be inspected directly while preparing this document. The program descriptions, code, flowcharts, and outputs below are illustrative examples inferred from the filenames, not verified copies of the repository's exact code. Run and compare each example with the original .py file before using this as a final practical record.

Contents

1. ATM Transaction Simulation (atm.py)
2. File Compression Demonstration (compression.py)
3. Duplicate Detection (duplicates.py)
4. Eligibility Checker (eligibility.py)
5. Food Ordering / Food Menu (food.py)
6. Inventory Management (inventory.py)
7. Matrix Operations (matrix.py)
8. Password Validation (password.py)
9. Pattern Printing (pattern.py)

How to use this record

Each activity includes its aim, step-by-step algorithm, a simplified flowchart, an example program, sample input, and expected output. The examples are intended to help explain the programming ideas and should be checked against the actual source files.

1. ATM Transaction Simulation (atm.py)

Pseudocode

```
START
Set the initial account balance.
Display the ATM transaction menu.
Read the user's choice.
If choice is balance enquiry, display balance.
If choice is deposit, read amount; if positive, add it to balance, otherwise display a
If choice is withdrawal, read amount; if valid and not greater than balance, subtract
If choice is invalid, display an error.
STOP
```

1. ATM Transaction Simulation (atm.py)

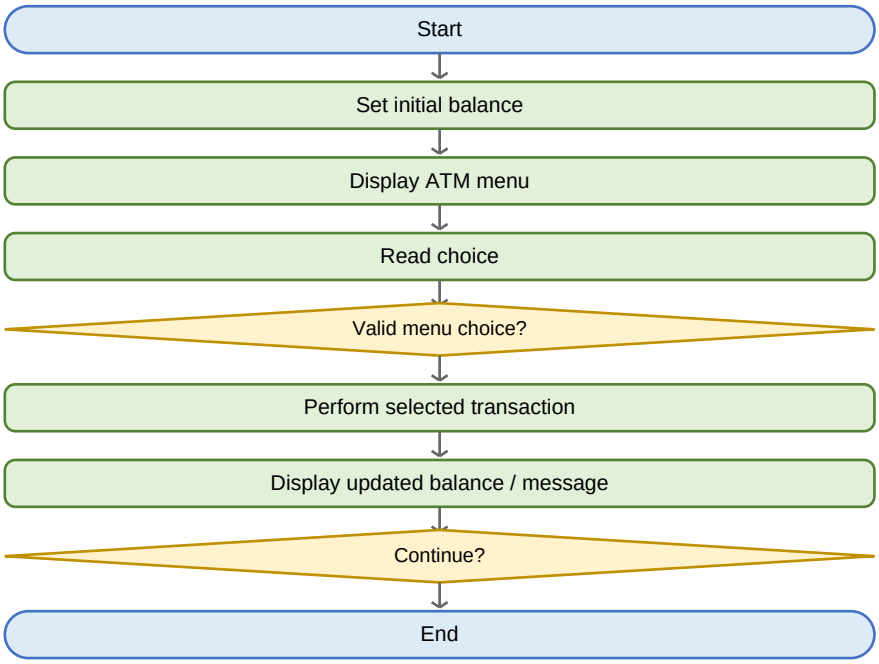
Aim

Simulate a simple ATM transaction using a menu, balance checks, and transaction choices.

Algorithm

1. Start the program.
2. Set an initial account balance.
3. Display transaction options such as balance enquiry, deposit, and withdrawal.
4. Read the user's choice and required amount.
5. Validate the amount and check available balance where relevant.
6. Update the balance and display the result.
7. Repeat for another transaction or exit.
8. Stop.

Flowchart (simplified)



Sample Python program

```
balance = 1000.0
print("ATM MENU")
print("1. Check balance")
print("2. Deposit")
print("3. Withdraw")
choice = input("Choose 1-3: ")

if choice == "1":
    print(f"Balance: Rs. {balance:.2f}")
elif choice == "2":
    amount = float(input("Deposit amount: "))
    if amount > 0:
        balance += amount
        print(f"Updated balance: Rs. {balance:.2f}")
    else:
        print("Amount must be positive")
elif choice == "3":
    amount = float(input("Withdrawal amount: "))
```

```
    if 0 < amount <= balance:
        balance -= amount
        print(f"Updated balance: Rs. {balance:.2f}")
    else:
        print("Invalid amount or insufficient balance")
else:
    print("Invalid choice")
```

Sample input

Choose 1-3: 3
Withdrawal amount: 250

Expected output

Updated balance: Rs. 750.00

Note: Illustrative example for a single transaction. The repository's exact source file was not available to inspect in this session; this example documents the activity suggested by the filename.

1. ATM Transaction Simulation (atm.py)

Conclusion

The ATM simulation demonstrates menu selection, conditional statements, and balance updates. It checks deposits and withdrawals before changing the account balance.

2. File Compression Demonstration (compression.py)

Pseudocode

```
START
Read or define the input string.
If the string is empty, return an empty result.
Initialize the result and set the first character count to 1.
Scan each character from left to right.
If the current character matches the previous character, increase the count.
Otherwise, append the previous character and its count; reset the count to 1.
Append the final character and its count.
Display the encoded string.
STOP
```

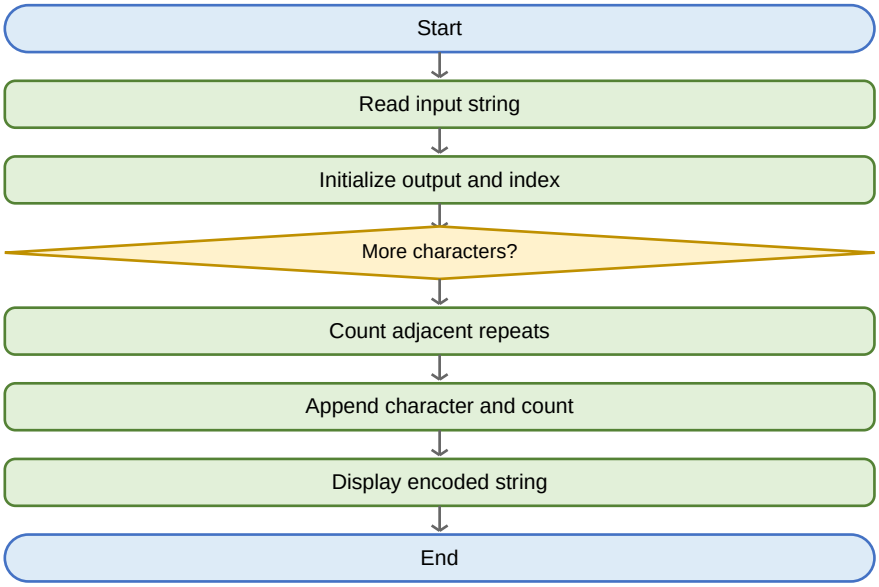
2. File Compression Demonstration (compression.py)

Aim
Demonstrate a basic lossless compression idea by encoding repeated adjacent characters as character-count pairs (run-length encoding).

Algorithm

1. Start.
2. Read a string.
3. Initialize an empty encoded result.
4. Scan the string from left to right, counting adjacent repeated characters.
5. Append each character and its count to the result.
6. Display the encoded result.
7. Stop.

Flowchart (simplified)



Sample Python program

```
def run_length_encode(text):
    if not text:
        return ""
    result = []
    count = 1
    for i in range(1, len(text)):
        if text[i] == text[i - 1]:
            count += 1
        else:
            result.append(text[i - 1] + str(count))
            count = 1
    result.append(text[-1] + str(count))
    return "".join(result)

sample = "aaabbc"
print("Original:", sample)
print("Encoded:", run_length_encode(sample))
```

Sample input

Sample string: aaabbc

Expected output

Original: aaabbc
Encoded: a3b2c1

Note: This is a teaching example of run-length encoding, not a claim about the exact algorithm in the repository's file.

2. File Compression Demonstration (compression.py)

Conclusion

The program demonstrates run-length encoding by replacing consecutive repeated characters with character-count pairs. The sample string 'aaabbc' is encoded as 'a3b2c1'.

3. Duplicate Detection (duplicates.py)

Pseudocode

```
START
Create or read a list of values.
Initialize an empty set named seen and an empty list named duplicates.
For each value in the list:
    If the value is already in seen and not yet in duplicates, add it to duplicates.
    Otherwise, add the value to seen.
Display the duplicate values.
STOP
```

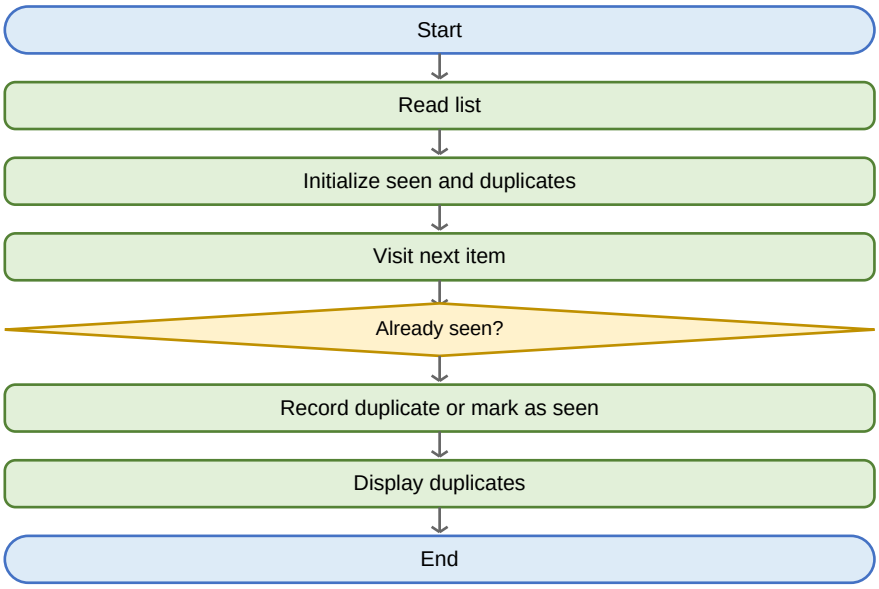
3. Duplicate Detection (duplicates.py)

Aim
Identify repeated values in a sequence.

Algorithm

1. Start.
2. Create or read a list of values.
3. Create an empty set for seen values and another collection for duplicates.
4. Visit each value in order.
5. If it has already been seen, record it as a duplicate; otherwise add it to seen.
6. Display duplicate values.
7. Stop.

Flowchart (simplified)



Sample Python program

```
values = [4, 2, 7, 2, 4, 9, 2]
seen = set()
duplicates = []

for value in values:
    if value in seen:
        if value not in duplicates:
            duplicates.append(value)
    else:
        seen.add(value)

print("Values:", values)
print("Duplicates:", duplicates)
```

Sample input

[4, 2, 7, 2, 4, 9, 2]

Expected output

Values: [4, 2, 7, 2, 4, 9, 2]
Duplicates: [2, 4]

3. Duplicate Detection (duplicates.py)

Conclusion

The program identifies repeated values using a set to track values already visited. Each duplicated value is listed only once, in the order it is first detected as a repeat.

Note: The example reports each duplicated value once, in the order it is first detected as a repeat.

4. Eligibility Checker (eligibility.py)

Pseudocode

```
START
Read the person's age.
If age is greater than or equal to 18, display 'Eligible'.
Otherwise, display 'Not eligible'.
STOP
```

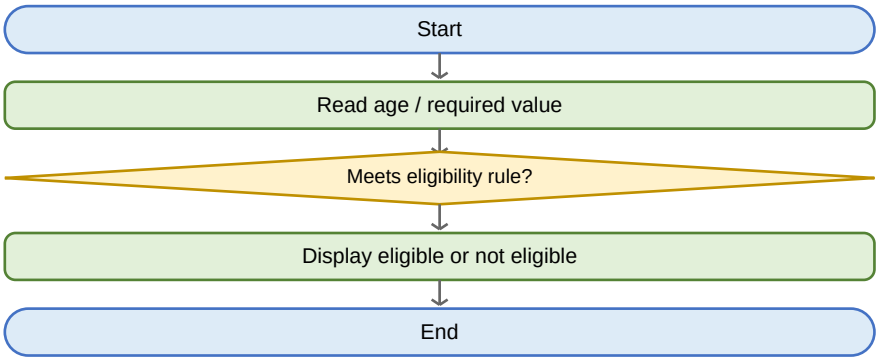
4. Eligibility Checker (eligibility.py)

Aim
Use conditional statements to decide whether a person meets a stated eligibility threshold.

Algorithm

1. Start.
2. Read the required input, such as age.
3. Check the input against the eligibility condition.
4. Display eligible or not eligible.
5. Stop.

Flowchart (simplified)



Sample Python program

```
age = int(input("Enter age: "))
if age >= 18:
    print("Eligible")
else:
    print("Not eligible")
```

Sample input

Enter age: 20

Expected output

Eligible

Note: The threshold is illustrative. Confirm the intended rule against the actual repository code before submission.

4. Eligibility Checker (eligibility.py)

Conclusion

The eligibility checker demonstrates decision-making with an if-else statement. The example uses 18 as an illustrative threshold.

5. Food Ordering / Food Menu (food.py)

Pseudocode

```
START
Create a menu containing food item numbers, names, and prices.
Display the available food items and prices.
Read the selected item number.
If the selection exists in the menu, display the item name and price.
Otherwise, display 'Invalid selection'.
STOP
```

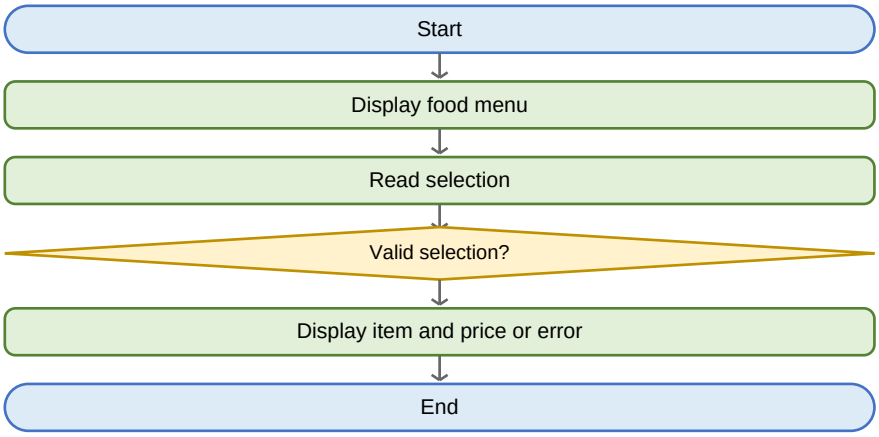
5. Food Ordering / Food Menu (food.py)

Aim
Display a food menu, accept a selection, and calculate the price of the chosen item.

Algorithm

1. Start.
2. Display available food items and prices.
3. Read the selected item number.
4. Check whether the selection is valid.
5. If valid, display the selected item and price; otherwise show an error.
6. Stop.

Flowchart (simplified)



Sample Python program

```
menu = {
    1: ("Idli", 30),
    2: ("Dosa", 50),
    3: ("Meals", 90)
}
print("1. Idli - Rs.30")
print("2. Dosa - Rs.50")
print("3. Meals - Rs.90")
choice = int(input("Select item: "))

if choice in menu:
    item, price = menu[choice]
    print(f"You selected {item}. Total: Rs.{price}")
else:
    print("Invalid selection")
```

Sample input

Select item: 2

Expected output

You selected Dosa. Total: Rs.50

Note: Example menu and prices are illustrative.

5. Food Ordering / Food Menu (food.py)

Conclusion

The food menu program demonstrates dictionaries and conditional selection. It displays the price for a valid item and reports an error for an invalid selection.

6. Inventory Management (inventory.py)

Pseudocode

```
START
Initialize the inventory with item names and quantities.
Read the item name.
If the item exists, display its current stock.
Read the quantity to add.
If the quantity is non-negative, update and display the stock.
Otherwise, display a quantity error.
If the item does not exist, display 'Item not found'.
STOP
```

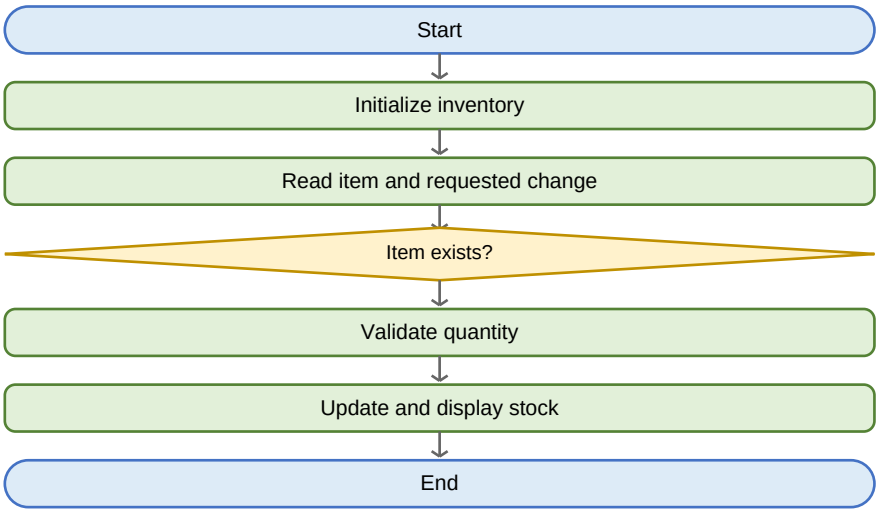
6. Inventory Management (inventory.py)

Aim
Maintain item quantities and support a simple stock lookup or update.

Algorithm

1. Start.
2. Initialize an inventory dictionary with item names and quantities.
3. Display or request an item name.
4. Check whether the item exists.
5. Read a quantity change if required and ensure the new quantity is not negative.
6. Update and display inventory.
7. Stop.

Flowchart (simplified)



Sample Python program

```
inventory = {"pen": 10, "notebook": 5, "pencil": 20}
item = input("Enter item: ").lower()

if item in inventory:
    print("Current stock:", inventory[item])
    change = int(input("Quantity to add: "))
    if change >= 0:
        inventory[item] += change
        print("Updated stock:", inventory[item])
    else:
        print("Quantity change must be non-negative")
else:
    print("Item not found")
```

Sample input

Enter item: pen
Quantity to add: 5

Expected output

Current stock: 10
Updated stock: 15

6. Inventory Management (inventory.py)

Conclusion

The inventory example demonstrates dictionary lookup and updating quantities. It validates the requested quantity before updating stock.

Note: This sample demonstrates stock lookup and adding quantity; it is not a verified reproduction of the repository's exact implementation.

7. Matrix Operations (matrix.py)

Pseudocode

```
START
Define or read two matrices of equal dimensions.
Create an empty result matrix.
For each row in the matrices:
    Create an empty result row.
    For each column, add the corresponding elements of both matrices.
    Append the result row to the result matrix.
Display each row of the result matrix.
STOP
```

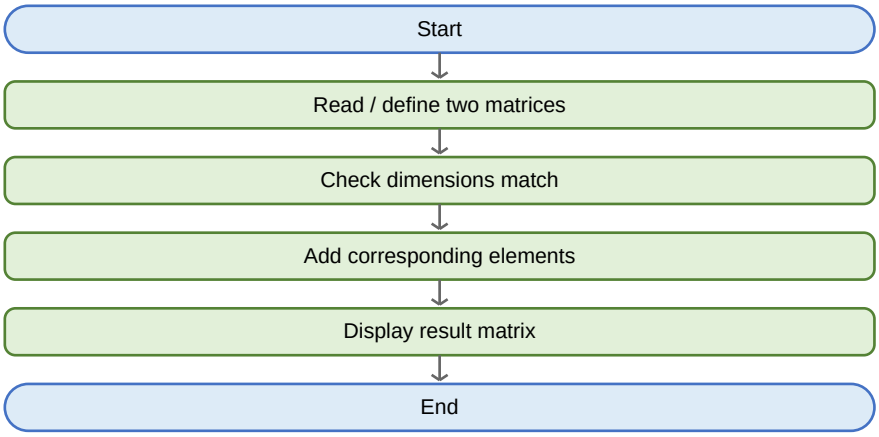
7. Matrix Operations (matrix.py)

Aim
Represent a matrix using nested lists and perform a basic operation such as matrix addition.

Algorithm

1. Start.
2. Define or read two matrices of equal dimensions.
3. Create an empty result matrix.
4. For each row and column, add corresponding elements.
5. Display the result matrix.
6. Stop.

Flowchart (simplified)



Sample Python program

```
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]
result = []

for i in range(len(A)):
    row = []
    for j in range(len(A[0])):
        row.append(A[i][j] + B[i][j])
    result.append(row)

print("Matrix sum:")
for row in result:
    print(row)
```

Sample input

A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]

Expected output

Matrix sum:
[6, 8]
[10, 12]

Note: Matrix addition is an illustrative activity; verify whether the source file performs addition or another matrix operation.

7. Matrix Operations (matrix.py)

Conclusion

The program adds corresponding elements of two matrices using nested loops. The result is displayed row by row.

8. Password Validation (password.py)

Pseudocode

```
START
Read the password.
Check whether its length is at least 8 characters.
Check whether it contains at least one digit.
Check whether it contains at least one uppercase letter.
If all checks pass, display that the password meets the sample rules.
Otherwise, display that it does not meet the sample rules.
STOP
```

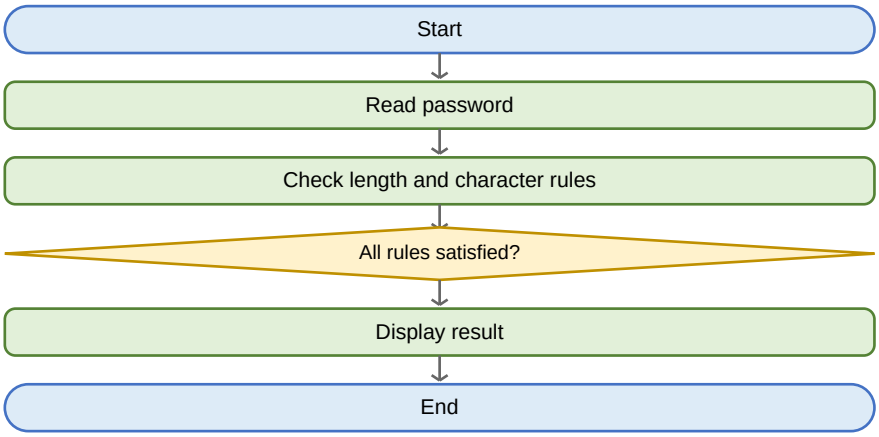
8. Password Validation (password.py)

Aim
Check whether a password meets a simple set of strength rules.

Algorithm

1. Start.
2. Read a password.
3. Check minimum length and the required character categories.
4. If all required checks pass, report valid/strong; otherwise report which requirements are missing.
5. Stop.

Flowchart (simplified)



Sample Python program

```
password = input("Enter password: ")
has_length = len(password) >= 8
has_digit = any(ch.isdigit() for ch in password)
has_upper = any(ch.isupper() for ch in password)

if has_length and has_digit and has_upper:
    print("Password meets the sample rules")
else:
    print("Password does not meet the sample rules")
```

Sample input

Enter password: LearnPython7

Expected output

Password meets the sample rules

Note: These are sample validation rules for demonstration only; use the source file's actual criteria if they differ.

8. Password Validation (password.py)

Conclusion

The password validator checks minimum length, the presence of a digit, and the presence of an uppercase letter. These are illustrative rules for this example.

9. Pattern Printing (pattern.py)

Pseudocode

```
START
Read the number of rows.
Set the row counter to 1.
While the row counter is less than or equal to the number of rows:
    Print a number of stars equal to the row counter.
    Increase the row counter by 1.
STOP
```

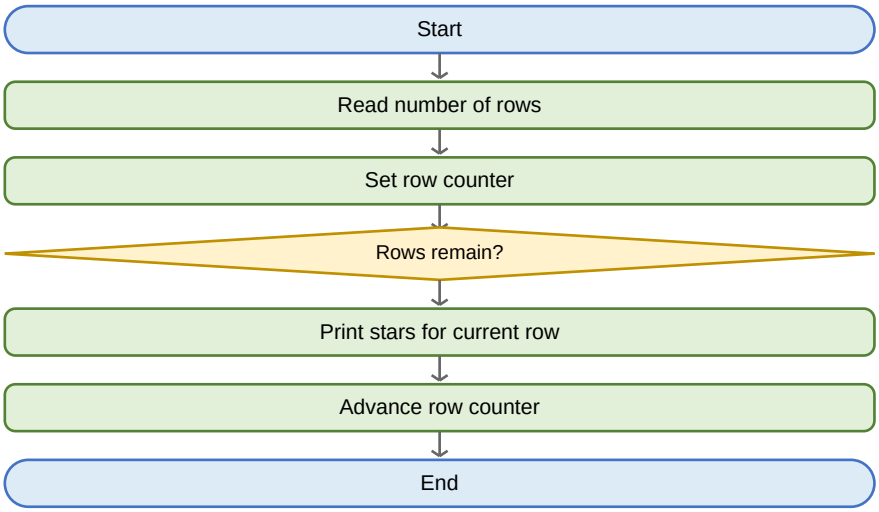
9. Pattern Printing (pattern.py)

Aim
Use nested loops to print a simple star pattern.

Algorithm

1. Start.
2. Read the number of rows.
3. For each row from 1 through the requested row count, print that many stars.
4. Move to the next line after each row.
5. Stop.

Flowchart (simplified)



Sample Python program

```
rows = int(input("Enter number of rows: "))
for i in range(1, rows + 1):
    print("*" * i)
```

Sample input

Enter number of rows: 4

Expected output

```
*
**
***
****
```

Note: A simple increasing triangle is shown; confirm the exact pattern expected by the repository's source file.

9. Pattern Printing (pattern.py)

Conclusion

The pattern-printing program uses a loop to print an increasing triangle of stars. Each row contains one more star than the previous row.

Conclusion

These activities illustrate fundamental Python skills: conditional logic, loops, lists and sets, dictionaries, string processing, input/output, and matrix representation. Algorithms help organize the solution before coding, flowcharts show the sequence of actions, and sample outputs make it easier to verify behavior.

Source repository

https://github.com/saikarthik-1736/Activity-Based-Learning-Python

For an exact repository-matched edition, provide the .py source files or upload a ZIP of the repository. Then the examples, algorithms, outputs, and flowcharts can be aligned with each original program.